



Rosbag格式分析



王方浩
自动驾驶话题下的优秀答主

关注他

36 人赞同了该文章 >

收起

这里主要参考Rosbag format 2.0，对Rosbag的格式做一个深度分析。

首先打开Rosbag，会在第一行标识出Rosbag的版本号，由于Rosbag有1.0, 1.1, 1.2, 2.0几个版本，通过版本号进行区分，这里我们只介绍最新的2.0版本。

版本号

首先打开Rosbag，会看到以下格式。也就是说Bag包的第一行是人眼可以识别的版本号，后面紧跟着的是一系列记录序列。

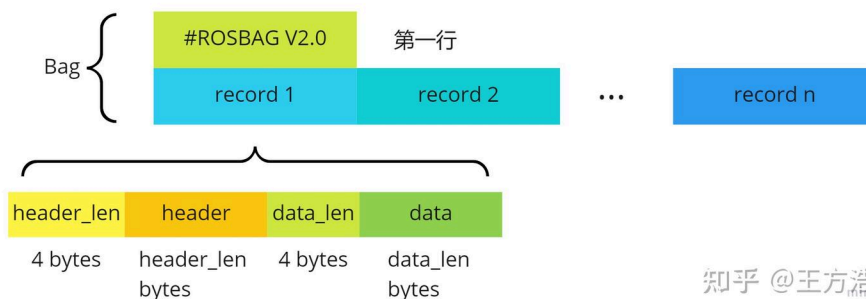
```
#ROSBAG V2.0
<record 1><record 2>....<record N>
```

记录 (Records)

上面已经知道了，Bag包由一系列的记录序列组成，接下来我们再详细的看每个记录的格式。

```
<header_len><header><data_len><data>
```

也就是说每个record由header和data组成，而为了找到header和data，还需要保存header_len和data_len，结合2者，我们可以得到Bag包的结构如下



知乎 @王方浩

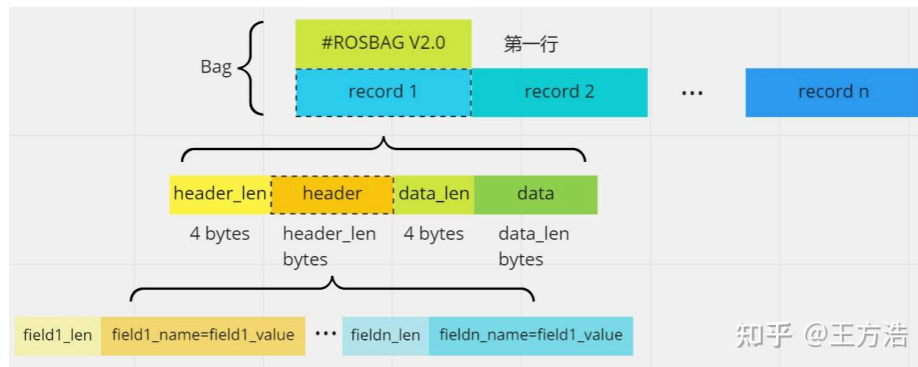
信息头 (Headers)



```
<field1_len><field1_name>=<field1_value><field2_len><field2_name>=<field2_value>
```

和 header_len 和 header 类似，这里的 field1_len 后面跟着 <field1_name>=<field1_value>，其中 field1_len 的长度包含 = 号在内，所有的 field 的长度等于 header_len。

- field_name 字段名称可以包含任何可打印的 ASCII 字符 (0x20 - 0x7e)，但 = (0x3d) 除外。
- field_value 字段值可以包含任何数据（包括嵌入空值、换行符等的二进制数据）



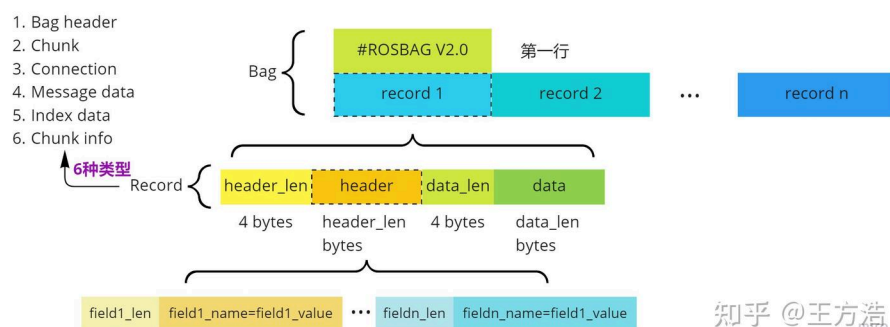
Op 码

所有的信息头必须包含Op码字段，也就是说上述 field_name 必须有一个字段为 Op。这个字段是为了区分 record 的类型而准备的，这也是为什么这个字段是必须的原因，因为没有这个字段就没法区分 record 的类型。从侧面也可以看出 header 信息头可以灵活的用来保存 record 的信息，甚至用户可以自定义一些信息。

不同的Op码对应不同的record类型，一共有6种类型，下面我们逐个介绍。

- **Bag header** 主要存放bag包整体的信息，必须是第一个record。
- **Chunk+** 主要的数据结构，可以被压缩，可以理解成把N个消息打包为一个块（Chunk），方便索引节省空间。
- **Connection+** 块的结构之一，存放信息的格式信息，有了消息的格式，才能解析消息。
- **Message data** 块的结构之一，消息序列化之后以2进制存储，通过Connection获取消息格式后进行反序列化。
- **Index data+** 索引数据，因为一个块比较大，索引消息在块中的位置，方便快捷查找，缺点在于会占用额外的空间
- **Chunk info+** 块的结构之一，主要描述块的信息，例如消息的起始和结束时间等。

至此Bag包的格式基本上就分析清楚了。接下来我们分别介绍这6种record的数据格式，也就是 record data 部分的内容。



6种记录类型

Bag header

- **index_pos** bag header之后第一条记录的偏移
- **conn_count** connections的数量
- **chunk_count** chunk的数量

Chunk

块，类似一个档案袋，打包了固定大小的多条消息。以下结构保存在header中。

- **compression** 压缩方式
- **size** 块大小

而data由Connection和Message data组成。

Connection

连接，通过连接可以获取消息的定义，用来反序列化消息，以下消息在header结构中

- **conn** 连接的id
- **topic** 消息的topic名称

数据段中包含

- **topic** 消息名称
- **type** 消息类型
- **md5sum** md5值
- **message_definition** 消息定义

Message data

消息本身，消息通过序列化保存，通过conn可以找到对应的连接（Connection），从而获取到消息类型，进行反序列化。

- **conn** 消息连接id，通过id可以找到对应的Connection
- **time** 消息发布时间

Index data

索引消息，主要是为了快速检索信息。

- **ver** 版本号
- **conn** 消息连接id
- **count** 消息数量

由于有多个消息，因此以下字段也会出现多次。

- **time** 时间
- **offset** 偏移

Chunk info

块信息，也是为了方便查找。

以下字段保存在header中

- **ver**
- **chunk_pos**
- **start_time**
- **end_time**

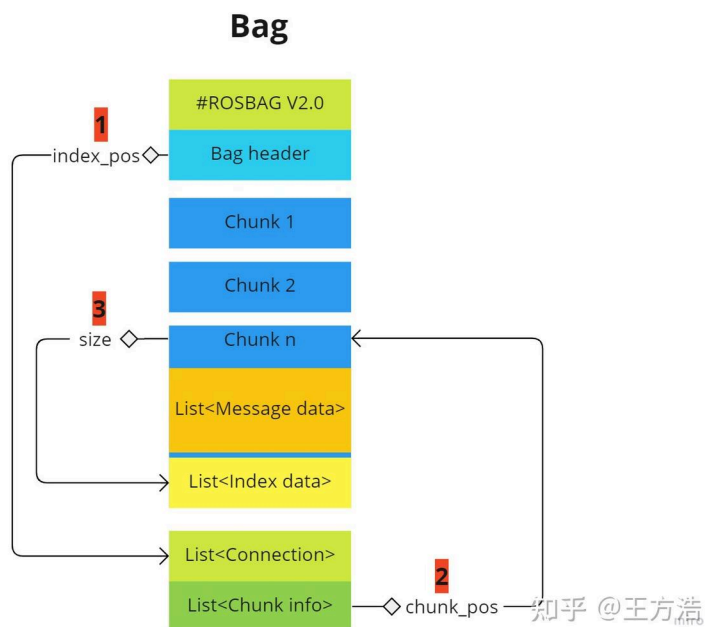
因为一个块中可能有多种不同的消息，因此以下2个字段会出现多次，类似一个哈希表的结构，保存在data结构中。

- conn
- count

读取顺序

上述6种类型的数据结构的读取顺序为：

1. 先解析Bag header，获取到index_pos
2. 然后跳到index_pos读取Connection
3. 接着读取Chunk info，上述2个步骤相当于建立起了整个Bag包，块的索引
4. 接着根据Chunk info逐个读取Chunk，先解析Chunk，获取压缩类型和数据大小
5. 接着跳到Chunk尾部解析Index data，这里是一个消息对应一个index
6. 最后根据index data实例化消息（通过接口instantiateBuffer）



设计原理

至此Bag包的分析就完成了，其实通过看上述的结构，第一遍可能只能有个大概的印象，为了进一步加深理解，这里我们对Bag的设计思路进行进一步的分析。

序列化和反序列化

首先我们知道bag包就是为了录制消息，而消息的保存和读取就涉及到一个广义上的问题**序列化和反序列化**，它基本上无处不在，只是大部分人没有注意到，举个很简单的例子，程序运行的时候，我们是直接操作的内存，也就是一个结构体或者一个对象，但内存里面的数据会消失，当我们要保存内存的数据到磁盘的时候就需要序列化之后保存，常见的序列化方式有XML、json等等，而protobuf也就是其中的一种。而当我们读取磁盘中的数据使用的时候，又需要把磁盘中的数据转换为内存中的数据，这个过程叫反序列化。

当然持久化并不是唯一利用到序列化和反序列化的一种，比如2个进程之间通信，由于进程之间的内存映射并不相同，也需要序列化和反序列化，同理还有2台机器之间的通信，例如最常见的网页应用也需要序列化和反序列化，当然还有更厉害的技术，例如跨语言的调用，制作一种通用的对象消息格式，从而实现不同语言之间的数据交换。

回到这个问题本身，也就是说持久化需要序列化消息，然后保存到硬盘，读取消息的时候，我们又需要反序列化消息为内存的对象。那么问题来了。

如果说写入消息很简单，但是读取消息的时候就麻烦了，因为你不知道消息的类型，就无法解析消息，假设你保存了3条消息，包括图片，位置和轨迹消息，你读取了一条消息，你怎么知道这条消息是什么类型呢？方法很简单，我们可以在消息头中标识这条消息是什么类型，然后再用这种消息类型去解析消息，这样就解决了。

通过消息类型名称（字符串）来生成对象，这在很多语言中叫做反射（reflection），这样我们就解决了消息解析的问题，而bag包中的Connection就是用来解决上述问题，它包含了数据类型和格式，而每个Message data中可以找到Connection，从而找到消息类型，进行解析。

还有一点设计的比较巧妙的地方在于，每个消息头如果都包含数据的消息定义的化，比较浪费空间，因此通过Connection中包含数据的格式（消息字段的定义），而Message data中只包含id，从而节省了空间。

索引

消息保存和解析的问题解决了，那么我们如何快速的查找呢？因为一个包可能比较大，所以把一个包拆分为几个块（Chunk），而chunk中有消息的时间段，这样查找指定时间段的消息的时候，就可以跳过一些块，从而避免读取整个块之后再进行查找，提高检索效率。

而Index data则更进一步，直接索引了不同类型的消息在块中的时间戳和偏移，从而方便快速查找。当然索引确实可以加快速度，但是过多的索引也会消耗空间，也就是经典的时间和空间的算法复杂度问题，需要取舍。

以上就是Rosbag的整个分析过程。

ROS Bag分析

- [rosvag record代码分析](#)
- [rosvag play代码分析](#)
- [rosvag_storage总体介绍（一）](#)
- [rosvag_storage bag实现（二）](#)
- [rosvag_storage 查询bag包（三）](#)
- [rosvag_storage 文件读写（四）](#)
- [Rosbag格式分析](#)

ROS 发布订阅模式

- [ROS发布订阅实现（二）](#)

ROS导航模块

- [ROS navigation导航模块（一）](#)

编辑于 2022-04-07 21:09

送礼物

还没有人送礼物，鼓励一下作者吧

内容所属专栏



自动驾驶
从零开始学习无人驾驶技术

订阅专栏

赞同 36 添加评论 分享 喜欢 收藏 申请转载 ...



理性发言，友善互动



还没有评论，发表第一个评论吧

推荐阅读

ROSBAG使用(一)：可视化bag文件

安装ROS可以参考：Win10安装ROS注：以下的命令行指的都是经过ROS的setup.bat的。首先，打开一个命令行，运行：shell roscore基本上，使用ROS的服务之前，都要先启动这个。然后， ...
Memory



判断文件及文件夹是否存在

VBA语言... 发表于VBA解决...

pandas筛选某列是否包含指定的字符串

```
import pandas as pd data = [{"name": "yyb", "age": 23}, {"name": "ays", "age": 25}]
```

亿安盐 发表于pytho...

ClickHouse为什么这么减少数据扫描范围

相信看过ClickHouse性能测的同学都很震惊于他超高的查询性能。于是下一步开始搜“ClickHouse性能为什么高”例如：列存储、数据压缩、理、向量化引擎 等等一些关张权-实时数据库